

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL411: COMPILER LAB

LAB MANUAL

PREPARED BY:

Ms. DELMY DAVID V

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

Program Educational Objectives (peos)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

- PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and Computer Science & Engineering specialization to the solution of complex engineering problems.
- PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.
- PO-10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

<i>Program: B. Tech</i>	<i>Department: Computer Science and Engineering</i>
<i>Course: Compiler Lab</i>	<i>Semester: S7 Batch: B Credits: 2</i>
<i>Course Code: CSL 411</i>	<i>Regulation: 2019</i>
<i>Course Area/Domain: Algorithms and Theory of algorithms</i>	L: 0 T: 0 TW/SL: 3 P: 3
Corresponding Lab Course Code (If Any): NA	<i>Lab Course Name: NA</i>

COURSE PRE-REQUISITES:

<i>CODE</i>	<i>COURSE NAME</i>	<i>DESCRIPTION</i>	<i>SEM</i>
CST 301	Formal languages and Automata Theory	Design automata, grammars, and parsers, understand the Chomsky hierarchy and decidability.	S5
CST 302	Compiler Design	Basic Understanding of Compiler phases and design	S6

COURSE OUTCOMES:

<i>SL NO</i>	<i>DESCRIPTION</i>
CO1	Implement the Lexical Analyzer using tool LEX
CO2	Implement Syntax Analyzer using tool YAAC
CO3	Design NFA and DFA for a problem and write programs to perform operations on it
CO4	Design and implement Top–Down Parsers and Bottom–Up Parsers
CO5	Implement intermediate code for expressions

CO - PO/PSO MAPPING:

CO	PO STRENGTH												PSO STRENGTH		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO 1	PSO2	PSO3
CO1	3	3	3	2	2			2		2		2		2	
CO2	3	3	3	2	2			2		2		2		2	
CO3	3	3	3	3				2		2		2		2	
CO4	3	3	3	3				2		2		2		2	
CO5	3	3	3	3				2		2		2		2	

GENERAL INSTRUCTIONS (for the lab)**Preparation:**

- Read the theory, algorithm, and expected output for the experiment scheduled before the lab session. Preparation is key to successful implementation.
- Familiarize yourself with the basic functionality of the Linux/Unix environment you will be using (e.g., shell navigation, basic command utilities).
- Bring your Lab Manual/Record to every session to document procedures, code snippets, and results in real-time.
- For experiments involving language/grammar design (e.g., writing a regular expression, designing a CFG, computing FIRST/FOLLOW sets), prepare the grammar or input specification on paper beforehand.

Lab Setup & Tools:

- Ensure you are logged into the assigned Linux/Unix machine or virtual environment.
- Familiarize yourself with the command-line editor (e.g., nano, vim) or IDE (e.g., VS Code, Eclipse) available for C/Java development.
- Confirm the availability and basic usage of the GNU Compiler Collection (gcc/g++) and, where required, LEX/Flex and YACC/Bison for compiling and generating your programs.
- Create a new directory for each experiment (e.g., mkdir exp_03_lexical_analyzer) to keep your source code, grammar files, and executables organized.

Coding Standards & Implementation:

- Language: All compiler construction tasks (Lexical Analysis, Parsing, Intermediate Code Generation) must be implemented using C, C++, or Java as specified by the instructor.
- Header Comments: Start every source file with a comment block including your name, roll number, date, experiment number, and the program's purpose.
- Grammar/Regex Files: When using LEX/YACC, clearly separate the specification file (.l/.y) from the driver/main code, and comment on each rule or production.
- Compilation Flags: Use the necessary flags for generating and linking lexer/parser code.
 - Example: flex lexer.l && gcc lex.yy.c -o lexer -lfl
 - Example: yacc -d parser.y && gcc y.tab.c -o parser -ly

- **Code Clarity:** Use clear, descriptive variable/function names for tokens, symbol tables, states, and grammar symbols.

Testing & Debugging:

- **Test Frequency:** Compile and test your code in small, incremental steps, especially after adding a new grammar rule or token definition.
- **Tracing:** Trace tokens, parse trees, or derivations manually for at least one test case before running the program, to verify correctness against your program's output.
- **Edge Cases:** Test your program with valid inputs, invalid/malformed inputs, and boundary cases (e.g., empty strings, nested expressions, ambiguous grammars) to confirm robust error handling.
- **Error Handling Check:** For parsers, ensure meaningful error messages are generated for syntax errors.(e.g., line number, expected vs. found token), not just abrupt program termination.

Documentation & Submission:

- **Demonstration:** Your code must be demonstrated live to the instructor or TA. Be prepared to explain the logic, algorithm, and role of key data structures (e.g., transition tables, parse stacks, symbol tables).
- **Viva Voce:** Be ready to answer questions regarding the underlying compiler theory concepts (e.g., difference between NFA and DFA, LL(1) vs LR(1) parsing, role of FIRST/FOLLOW sets).
- **Lab Record:** All components (Aim, Theory, Algorithm, Code, Procedure, Observed Result) must be formally entered into your record book.
- **Submission:** Submit the requested source files (.c, .cpp, .l, .y) and the output/result sheet according to the instructor's specifications (e.g., file naming convention: UCSEST105_ExptX_RollNo.c).

Safety & Conduct:

- **System Safety:** NEVER attempt to run system calls or commands that could compromise the stability or security of the operating system.
- **System Login:** Always log off your account before leaving the lab.
- **Ask for Help:** If you encounter persistent compilation errors, grammar conflicts (shift-reduce/reduce-reduce), or unexpected parser behavior, ask the instructor immediately

COURSE SYLLABUS**Lab Experiments:**

1. Design and implement a lexical analyzer using C language to recognize all valid tokens in the input program. The lexical analyzer should ignore redundant spaces, tabs and newlines. It should also ignore comments.
2. Implement a Lexical Analyzer for a given program using Lex Tool.
3. Write a lex program to display the number of lines, words and characters in an input text.
4. Write a LEX Program to convert the substring *abc* to *ABC* from the given input string.
5. Write a lex program to find out the total number of vowels and consonants from the given input string.
6. Generate a YACC specification to recognize a valid arithmetic expression that uses operators +, -, *, / and parenthesis.
7. Generate a YACC specification to recognize a valid identifier which starts with a letter followed by any number of letters or digits.
8. Implementation of Calculator using LEX and YACC
9. Convert the BNF rules into YACC form and write code to generate abstract syntax trees.
10. Write a program to find ϵ – closure of all states of any given NFA with ϵ transition.
11. Write a program to convert NFA with ϵ transition to NFA without ϵ transition.
12. Write a program to convert NFA to DFA.
13. Write a program to minimize any given DFA.
14. Write a program to find first and Follow of any given grammar.
15. Design and implement a recursive descent parser for a given grammar.
16. Construct a Shift Reduce Parser for a given language.
17. Write a program to perform constant propagation.
18. Implement Intermediate code generation for simple expressions.
19. Implement the back end of the compiler which takes the three address codes and produces the 8086 assembly language instructions that can be assembled and run using an 8086 assembler. The target assembly instructions can be simple move, add, sub, jump etc.

INDEX

Exp No.	Experiment Name	Page Number
1	LEXICAL ANALYZER FOR A C PROGRAM	10
2	IMPLEMENTATION OF A LEXICAL ANALYZER FOR A GIVEN PROGRAM USING LEX TOOL	23
3	DISPLAY THE NUMBER OF LINES, WORDS AND CHARACTERS IN AN INPUT TEXT USING LEX	30
4	CONVERT THE SUBSTRING "abc" TO "ABC" USING LEX	34
5	LEX PROGRAM TO COUNT VOWELS AND CONSONANTS IN A GIVEN STRING	37
6	RECOGNIZE A VALID ARITHMETIC EXPRESSION USING YACC	40
7	RECOGNIZE A VALID IDENTIFIER USING YACC	45
8	IMPLEMENTATION OF A CALCULATOR USING LEX AND YACC	50
9	CONVERSION OF BNF RULES TO YACC FORM AND GENERATION OF ABSTRACT SYNTAX TREES	56

10	COMPUTATION OF EPSILON-CLOSURE OF ALL STATES OF A GIVEN NFA WITH EPSILON TRANSITIONS	63
11	CONVERSION OF NFA WITH EPSILON TRANSITIONS TO NFA WITHOUT EPSILON TRANSITIONS	68
12	CONVERSION OF NFA TO DFA (SUBSET CONSTRUCTION)	73
13	MINIMIZATION OF A GIVEN DFA	77
14	COMPUTATION OF FIRST AND FOLLOW OF A GIVEN GRAMMAR	83
15	DESIGN AND IMPLEMENTATION OF A RECURSIVE DESCENT PARSER	90
16	CONSTRUCTION OF A SHIFT-REDUCE PARSER FOR A GIVEN LANGUAGE	96
17	IMPLEMENTATION OF CONSTANT PROPAGATION	100
18	INTERMEDIATE CODE GENERATION FOR SIMPLE EXPRESSIONS	107
19	IMPLEMENTATION OF THE BACK END OF A COMPILER — TAC TO 8086 ASSEMBLY	112

EXP NO: 01

LEXICAL ANALYZER FOR A C PROGRAM

AIM:

To design and implement a lexical analyzer using the C language that recognizes all the valid tokens present in an input program. The lexical analyzer should ignore redundant spaces, tabs and newlines, and should also ignore comments.

LEARNING OUTCOMES:

Upon completion of this exercise, the student will be able to:

1. Understand the role of the lexical analysis phase in the design of a compiler.
2. Classify the lexemes of a source program into tokens such as keywords, identifiers, constants, operators, special symbols and string literals.
3. Design and trace a finite-automaton-based scanning logic for token recognition.
4. Implement file handling and character-by-character stream processing in C.
5. Filter out whitespace (blanks, tabs, newlines) and both single-line and multi-line comments before tokenization.

DESCRIPTION:

Lexical analysis is the first phase of a compiler. The lexical analyzer (also called a scanner) reads the source program character by character, groups the characters into meaningful sequences called lexemes, and produces a token for each lexeme in the form <token-type, lexeme/attribute-value>. A token represents a logical unit such as a keyword, identifier, constant, operator or punctuation symbol. The lexical analyzer also strips away elements that carry no semantic meaning for later phases of compilation, namely redundant white space (blanks, tabs, newlines) and comments, and it detects the boundaries of one lexeme from the next using the principle of maximal munch, that is, it always matches the longest possible string of characters that forms a valid token.

The lexical analyzer designed here recognizes the following categories of tokens:

- Keywords: reserved words of the C language such as int, float, if, else, while, for, return, etc.

- Identifiers: user-defined names for variables and functions, beginning with a letter or underscore followed by letters, digits or underscores.
- Constants: integer constants and floating point (real) constants.
- String literals: sequences of characters enclosed within double quotes.
- Operators: arithmetic, relational, logical and assignment operators, including two-character operators such as ==, !=, <=, >=, &&, ||, ++ and --.
- Special symbols / punctuators: (,), {, }, [,], ; and ,

Two kinds of comments are recognized and discarded: single-line comments beginning with // and continuing till the end of the line, and multi-line comments delimited by /* and */ (spelt with an asterisk immediately followed by a slash), which may span several lines.

ALGORITHM:

1. Start. Open the source program file in read mode.
2. Read the file character by character until the end of the file.
3. Skip blanks, tabs and newlines.
4. Skip single-line (//) and multi-line (/* */) comments.
5. If a letter/underscore is found, read the full word and classify it as KEYWORD or IDENTIFIER.
6. If a digit is found, read the full number and classify it as an INTEGER or FLOAT CONSTANT.
7. If a double quote is found, read till the closing quote and classify it as a STRING LITERAL.
8. If an operator symbol is found, check for a possible two-character operator and classify it as OPERATOR.
9. If a bracket, brace or punctuation mark is found, classify it as a SPECIAL SYMBOL.
10. Print each token with its token type.
11. Close the file. Stop.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX 100

/* List of C keywords recognized by the analyzer */
```

```
char *keywords[] = {  
  
    "int", "float", "char", "double", "void", "if", "else", "while",  
  
    "for", "do", "switch", "case", "break", "continue", "return",  
  
    "struct", "typedef", "const", "static", "long", "short", "signed",  
  
    "unsigned", "default", "sizeof", "goto"  
  
};  
  
#define NUM_KEYWORDS (sizeof(keywords) / sizeof(keywords[0]))  
  
int isKeyword(char *buffer) {  
  
    int i;  
  
    for (i = 0; i < NUM_KEYWORDS; i++)  
  
        if (strcmp(buffer, keywords[i]) == 0)  
  
            return 1;  
  
    return 0;  
  
}  
  
int main() {  
  
    FILE *fp;  
  
    char ch, buffer[MAX];  
  
    int i = 0, lineNo = 1;  
  
    char filename[100];  
  
    printf("Enter the source program file name: ");  
  
    scanf("%s", filename);  
  
    fp = fopen(filename, "r");
```

```
if (fp == NULL) {

printf("Unable to open the file.\n");

return 0;

}

printf("\n%-20s%-20s\n", "TOKEN", "TOKEN TYPE");

printf("-----\n");

ch = fgetc(fp);

while (ch != EOF) {

/* Skip blanks and tabs */

if (ch == ' ' || ch == '\t') {

ch = fgetc(fp);

continue;

}

/* Skip newlines, but track the line number */

if (ch == '\n') {

lineNo++;

ch = fgetc(fp);

continue;

}

/* Skip comments: // ... and /* ... *(slash) */

if (ch == '/') {

char next = fgetc(fp);
```

```
if (next == '/') {  
  
while (ch != '\n' && ch != EOF)  
  
ch = fgetc(fp);  
  
continue;  
  
}  
  
else if (next == '*') {  
  
char prev = 0;  
  
ch = fgetc(fp);  
  
while (!(prev == '*' && ch == '/') && ch != EOF) {  
  
prev = ch;  
  
ch = fgetc(fp);  
  
}  
  
ch = fgetc(fp);  
  
continue;  
  
}  
  
else {  
  
printf("%-20c%-20s\n", ch, "OPERATOR");  
  
ch = next;  
  
continue;  
  
}  
  
}  
  
/* Identifiers and keywords */
```

```
if (isalpha(ch) || ch == '_') {

i = 0;

while (isalnum(ch) || ch == '_') {

buffer[i++] = ch;

ch = fgetc(fp);

}

buffer[i] = '\0';

if (isKeyword(buffer))

printf("%-20s%-20s\n", buffer, "KEYWORD");

else

printf("%-20s%-20s\n", buffer, "IDENTIFIER");

continue;

}

/* Numeric constants (integer and float) */

if (isdigit(ch)) {

i = 0;

int isFloat = 0;

while (isdigit(ch) || ch == '.') {

if (ch == '.') isFloat = 1;

buffer[i++] = ch;

ch = fgetc(fp);

}
```

```
buffer[i] = '\0';

printf("%-20s%-20s\n", buffer,

isFloat ? "FLOAT CONSTANT" : "INTEGER CONSTANT");

continue;

}

/* String literals */

if (ch == '"') {

i = 0;

buffer[i++] = ch;

ch = fgetc(fp);

while (ch != '"' && ch != EOF) {

buffer[i++] = ch;

ch = fgetc(fp);

}

buffer[i++] = ch;

buffer[i] = '\0';

printf("%-20s%-20s\n", buffer, "STRING LITERAL");

ch = fgetc(fp);

continue;

}

/* Two-character operators: ==, !=, <=, >=, &&, ||, ++, -- */

if (strchr("=!<>&|+-", ch)) {
```

```
char next = fgetc(fp);

char op2[3] = {ch, next, '\0'};

if ((ch == '=' && next == '=') || (ch == '!' && next == '=') ||

(ch == '<' && next == '=') || (ch == '>' && next == '=') ||

(ch == '&' && next == '&') || (ch == '|' && next == '|') ||

(ch == '+' && next == '+') || (ch == '-' && next == '-')) {

printf("%-20s%-20s\n", op2, "OPERATOR");

ch = fgetc(fp);

} else {

printf("%-20c%-20s\n", ch, "OPERATOR");

ch = next;

}

continue;

}

/* Remaining single character operators */

if (strchr("*/%", ch)) {

printf("%-20c%-20s\n", ch, "OPERATOR");

ch = fgetc(fp);

continue;

}

/* Special symbols / punctuators */

if (strchr("{}[];,", ch)) {
```

```
printf("%-20c%-20s\n", ch, "SPECIAL SYMBOL");

ch = fgetc(fp);

continue;

}

/* Any other character (e.g. '#') is simply skipped */

ch = fgetc(fp);

}

fclose(fp);

return 0;

}
```

PROCEDURE:

1. Type the source code of the lexical analyzer given above and save it as lexer.c.
2. Prepare a sample C source file (for example, test.c) whose tokens are to be identified; the sample file may contain keywords, identifiers, constants, operators, special symbols, string literals and both types of comments.
3. Compile the lexical analyzer: gcc lexer.c -o lexer
4. Run the compiled program: ./lexer
5. When prompted, enter the name of the source file to be scanned, for example test.c.
6. Observe the list of tokens printed by the program along with their token types, and verify that comments and redundant white space have been correctly ignored.

SAMPLE INPUT (test.c):

```
/* Program to find the sum of two numbers */

#include <stdio.h>

int main()

{
```

```
int a = 10, b = 20; // declare two integers

int sum;

sum = a + b;

if (sum >= 30)

{

printf("Sum is %d\n", sum);

}

return 0;

}
```

SAMPLE OUTPUT:

Enter the source program file name: test.c

TOKEN TOKEN TYPE

include IDENTIFIER

< OPERATOR

stdio IDENTIFIER

h IDENTIFIER

> OPERATOR

int KEYWORD

main IDENTIFIER

(SPECIAL SYMBOL

) SPECIAL SYMBOL

```
{ SPECIAL SYMBOL  
  
int KEYWORD  
  
a IDENTIFIER  
  
= OPERATOR  
  
10 INTEGER CONSTANT  
  
, SPECIAL SYMBOL  
  
b IDENTIFIER  
  
= OPERATOR  
  
20 INTEGER CONSTANT  
  
; SPECIAL SYMBOL  
  
int KEYWORD  
  
sum IDENTIFIER  
  
; SPECIAL SYMBOL  
  
sum IDENTIFIER  
  
= OPERATOR  
  
a IDENTIFIER  
  
+ OPERATOR  
  
b IDENTIFIER  
  
; SPECIAL SYMBOL  
  
if KEYWORD  
  
( SPECIAL SYMBOL  
  
sum IDENTIFIER
```

>= OPERATOR

30 INTEGER CONSTANT

) SPECIAL SYMBOL

{ SPECIAL SYMBOL

printf IDENTIFIER

(SPECIAL SYMBOL

"Sum is %d\n" STRING LITERAL

, SPECIAL SYMBOL

sum IDENTIFIER

) SPECIAL SYMBOL

; SPECIAL SYMBOL

} SPECIAL SYMBOL

return KEYWORD

0 INTEGER CONSTANT

; SPECIAL SYMBOL

} SPECIAL SYMBOL

MODEL VIVA QUESTIONS:

1. What is lexical analysis and why is it the first phase of a compiler?
2. What is a token, a lexeme and a pattern? Explain with an example.
3. What is the difference between a token and a lexeme?
4. What is meant by the 'maximal munch' (longest match) rule?
5. How does the lexical analyzer distinguish a keyword from an identifier?

RESULT:

A lexical analyzer to recognize valid tokens in a C program was designed and implemented successfully, and the output was verified.

EXP NO: 2

IMPLEMENTATION OF A LEXICAL ANALYZER FOR A GIVEN PROGRAM **USING LEX TOOL**

AIM:

To implement a lexical analyzer for a given C program using the Lex tool, which recognizes keywords, identifiers, constants, operators, special symbols and string literals, while ignoring white space and comments.

LEARNING OUTCOMES:

Upon completion of this exercise, the student will be able to:

1. Understand the structure of a Lex specification file and its three sections.
2. Write regular expressions to specify the patterns of different token classes.
3. Use the Lex-generated scanner (lex.yy.c) along with a driver program to build a working lexical analyzer.
4. Compile and run a Lex program using the flex and gcc toolchain.
5. Compare automatic (Lex-based) lexical analysis with a hand-written scanner.

DESCRIPTION:

Lex (and its GNU version, Flex) is a tool that automatically generates a lexical analyzer from a set of regular expressions supplied by the programmer. A Lex source file, conventionally given the extension .l, consists of three sections separated by the delimiter %%:

- Definitions section: contains C declarations and header includes enclosed in %{ ... %}, along with optional macro (regular) definitions.
- Rules section: contains a list of regular expressions, each followed by a C action that is executed whenever the input matches that expression. The predefined variable yytext holds the matched lexeme.
- User code section: contains the main() function that calls yylex() to start scanning, and any other supporting C functions such as yywrap().

When run through the Lex compiler (flex lexer.l), the tool translates the specification into a C source file, lex.yy.c, containing the function yylex(). This file is then compiled with a standard C compiler (and linked with the Lex library) to obtain the final lexical analyzer. The generated scanner reads the input and, for each token, executes the action associated with the longest matching rule (the maximal-munch principle), which makes it well suited for identifying keywords, identifiers, constants, string literals, operators and special symbols, while discarding white space and comments.

ALGORITHM:

1. Start.
2. Write a Lex specification file (lexer.l) with definitions, rules and user code sections.
3. In the rules section, define patterns to skip single-line and multi-line comments, and white space.
4. Define a pattern that matches C keywords and prints the token type KEYWORD.
5. Define a pattern that matches identifiers and prints the token type IDENTIFIER.
6. Define patterns that match integer and floating point constants.
7. Define a pattern that matches string literals enclosed in double quotes.
8. Define patterns for two-character and single-character operators.
9. Define a pattern that matches special symbols/punctuators.
10. Compile the specification using flex to generate lex.yy.c.
11. Compile lex.yy.c with gcc, linking the Lex library, to obtain the executable.
12. Run the executable on a sample source file and observe the tokens printed. Stop.

PROGRAM (lexer.l):

```
%{  
  
#include <stdio.h>  
  
#include <string.h>  
  
int lineNo = 1;  
  
%}  
  
%%  
  
"/*".* { /* skip single-line comment */ }  
  
"/*"([^\*]|\\*+[^*/])*\*+/" { /* skip multi-line comment */ }
```

```

"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"do"|"switch"|"case"|"break"|"continue"|"return"|"struct"|"typedef"|"const"|"static"|"long"|"short"|"signed"|"unsigned"|"default"|"sizeof"|"goto"

{ printf("%-20s%-20s\n", yytext, "KEYWORD"); }

[a-zA-Z_][a-zA-Z0-9_]* { printf("%-20s%-20s\n", yytext, "IDENTIFIER"); }

[0-9]+\.[0-9]+ { printf("%-20s%-20s\n", yytext, "FLOAT CONSTANT"); }

[0-9]+ { printf("%-20s%-20s\n", yytext, "INTEGER CONSTANT"); }

\"([^\"]|\\.)*\\" { printf("%-20s%-20s\n", yytext, "STRING LITERAL"); }

"=="|"!="|"<="|">="|"&&"|"||"|"++"|"--" { printf("%-20s%-20s\n", yytext, "OPERATOR"); }

[-+*/%=<>!&] { printf("%-20s%-20s\n", yytext, "OPERATOR"); }

[(){}\\[\\];,] { printf("%-20s%-20s\n", yytext, "SPECIAL SYMBOL"); }

\\n { lineNo++; }

[ \\t]+ { /* skip whitespace */ }

. { /* ignore any other character, e.g. '#' */ }

%%

int main(int argc, char *argv[])

{

if (argc > 1) {

yyin = fopen(argv[1], "r");

if (!yyin) {

printf("Cannot open file %s\n", argv[1]);

return 0;

}

}

```

```
}

printf("%-20s%-20s\n", "TOKEN", "TOKEN TYPE");

printf("-----\n");

yylex();

return 0;

}

int yywrap()

{

return 1;

}
```

PROCEDURE:

1. Type the Lex specification given above and save it as lexer.l.
2. Prepare a sample C source file (for example, test.c) whose tokens are to be identified.
3. Generate the C scanner from the specification: flex lexer.l (this produces lex.yy.c).
4. Compile the generated scanner: gcc lex.yy.c -o lexer -lfl
5. Run the executable, passing the sample file as an argument: ./lexer test.c
6. Observe the tokens printed along with their token types, and verify that comments and white space have been correctly ignored.

SAMPLE INPUT (test.c):

```
/* Program to find the sum of two numbers */

#include <stdio.h>

int main()

{

int a = 10, b = 20; // declare two integers
```

```
int sum;

sum = a + b;

if (sum >= 30)

{

printf("Sum is %d\n", sum);

}

return 0;

}
```

SAMPLE OUTPUT:

TOKEN TOKEN TYPE

include IDENTIFIER

< OPERATOR

stdio IDENTIFIER

h IDENTIFIER

> OPERATOR

int KEYWORD

main IDENTIFIER

(SPECIAL SYMBOL

) SPECIAL SYMBOL

{ SPECIAL SYMBOL

int KEYWORD

a IDENTIFIER

= OPERATOR

10 INTEGER CONSTANT

, SPECIAL SYMBOL

b IDENTIFIER

= OPERATOR

20 INTEGER CONSTANT

; SPECIAL SYMBOL

int KEYWORD

sum IDENTIFIER

; SPECIAL SYMBOL

sum IDENTIFIER

= OPERATOR

a IDENTIFIER

+ OPERATOR

b IDENTIFIER

; SPECIAL SYMBOL

if KEYWORD

(SPECIAL SYMBOL

sum IDENTIFIER

>= OPERATOR

30 INTEGER CONSTANT

```
) SPECIAL SYMBOL  
  
{ SPECIAL SYMBOL  
  
printf IDENTIFIER  
  
( SPECIAL SYMBOL  
  
"Sum is %d\n" STRING LITERAL  
  
, SPECIAL SYMBOL  
  
sum IDENTIFIER  
  
) SPECIAL SYMBOL  
  
; SPECIAL SYMBOL  
  
} SPECIAL SYMBOL  
  
return KEYWORD  
  
0 INTEGER CONSTANT  
  
; SPECIAL SYMBOL  
  
} SPECIAL SYMBOL
```

MODEL VIVA QUESTIONS:

1. What is Lex? What is the difference between Lex and Flex?
2. What are the three sections of a Lex program, and what does each contain?
3. What is the purpose of the yylex() function?
4. What does the variable yytext represent?
5. What is the role of yywrap() in a Lex program?

RESULT:

A lexical analyzer for a given C program was implemented successfully using the Lex tool, and the output was verified.

EXP NO: 3

DISPLAY THE NUMBER OF LINES, WORDS AND CHARACTERS IN AN INPUT TEXT USING LEX

AIM:

To write a Lex program that counts the number of lines, words, and characters present in an input text file.

LEARNING OUTCOMES:

Upon completion of this experiment, the student will be able to:

1. Understand how Lex processes input using pattern matching.
2. Use Lex rules to count lines, words, and characters.
3. Generate a lexical analyzer using the Flex tool.
4. Compile and execute a Lex program using Flex and GCC.
5. Analyze text files using Lex-generated scanners.

DESCRIPTION:

Lex (Flex) is a lexical analyzer generator that recognizes input patterns using regular expressions. In this experiment, Lex scans an input text character by character. Every character is counted to determine the total number of characters. Whenever a sequence of letters or digits separated by spaces or punctuation is encountered, it is counted as a word. Similarly, each newline character ('\n') is counted as a line. After scanning the entire input, the program displays the total number of lines, words, and characters.

ALGORITHM:

1. Start.
2. Initialize line, word, and character counters to zero.
3. Read the input text.
4. Count every character read.
5. Count every word using an appropriate regular expression.

6. Count every newline character.
7. Continue until the end of the input.
8. Display the number of lines, words, and characters.
9. Stop.

PROGRAM (count.l):

```
%{  
  
#include<stdio.h>  
  
int lines=0, words=0, chars=0;  
  
%}  
  
%%  
  
\n      { lines++; chars++; }  
  
[ \t]+  { chars += yyleng; }  
  
[^ \t\n]+ { words++; chars += yyleng; }  
  
.      { chars++; }  
  
%%  
  
int main()  
{  
  
    printf("Enter the text (Press Ctrl+D to end):\n");  
  
    yylex();  
  
    printf("\nNumber of Lines    : %d", lines);  
  
    printf("\nNumber of Words      : %d", words);  
  
    printf("\nNumber of Characters : %d\n", chars);  
  
    return 0;
```

```
}  
  
int yywrap()  
{  
  
    return 1;  
  
}
```

PROCEDURE:

1. Type the above Lex program and save it as count.l.
2. Open the terminal.
3. Generate the scanner using:
 - a. flex count.l
4. Compile the generated C file.
 - . gcc lex.yy.c -o count -lfl
5. Execute the program.
 - . ./count
6. Enter the input text.
7. Press Ctrl+D to terminate the input.
8. Observe the displayed number of lines, words, and characters.

SAMPLE INPUT:

Lex is a scanner generator.

It recognizes tokens.

SAMPLE OUTPUT:

Number of Lines : 2

Number of Words : 7

Number of Characters : 46

MODEL VIVA QUESTIONS:

1. What is Lex?
2. What is Flex?
3. What is a lexical analyzer?
4. What is the purpose of yylex()?
5. What does yywrap() do?

RESULT:

The Lex program to count the number of lines, words, and characters in an input text was implemented successfully, and the output was verified.

EXP NO: 4

CONVERT THE SUBSTRING "abc" TO "ABC" USING LEX

AIM:

To write a Lex program that converts every occurrence of the substring **abc** into **ABC** in the given input string.

LEARNING OUTCOMES:

Upon completion of this experiment, the student will be able to:

1. Understand string substitution using Lex.
2. Apply regular expressions for pattern matching.
3. Replace matched patterns with desired output.
4. Generate and execute a Lex scanner.
5. Perform simple text processing using Lex.

DESCRIPTION:

Lex is widely used for pattern matching and text processing. In this experiment, the scanner searches the input for every occurrence of the substring abc. Whenever the pattern is matched, the corresponding action prints ABC instead. All other characters are printed without modification. Thus, the output string is identical to the input except that every occurrence of abc is replaced with ABC.

ALGORITHM:

1. Start.
2. Read the input string.
3. Search for the pattern abc.
4. If the pattern matches, print ABC.
5. Otherwise, print the original character.
6. Continue until the end of the input.

7. Stop.

PROGRAM (replace.l):

```
%{  
  
#include<stdio.h>  
  
%}  
  
%%  
  
abc      { printf("ABC"); }  
  
\n      { printf("\n"); }  
  
.        { printf("%s", yytext); }  
  
%%  
  
int main()  
  
{  
  
    printf("Enter the input string:\n");  
  
    yylex();  
  
    return 0;  
  
}  
  
int yywrap()  
  
{  
  
    return 1;  
  
}
```

PROCEDURE:

1. Type the Lex program and save it as replace.l.
2. Open the terminal.

3. Generate the scanner.

```
flex replace.l
```

4. Compile the generated C program.

```
gcc lex.yy.c -o replace -lfl
```

5. Execute the program.

```
./replace
```

6. Enter the input string.

7. Press Ctrl+D to end the input.

8. Observe the converted output.

SAMPLE INPUT:

abc is replaced by abc in abcxyz.

SAMPLE OUTPUT:

ABC is replaced by ABC in ABCxyz.

MODEL VIVA QUESTIONS:

1. What is Lex?
2. What is a regular expression?
3. How does Lex perform pattern matching?
4. What is yytext?
5. What is the purpose of yylex()?

RESULT:

The Lex program to replace every occurrence of the substring abc with ABC was implemented successfully, and the output was verified.

EXP NO: 5

LEX PROGRAM TO COUNT VOWELS AND CONSONANTS IN A GIVEN STRING

AIM:

To write a Lex program that reads an input string and counts the total number of vowels and consonants present in it.

ALGORITHM:

1. Define a Lex specification with the usual three sections: definitions, translation rules, and auxiliary C functions.
2. In the rules section, specify a pattern that matches any vowel (a, e, i, o, u, in either case) and increments a vowel counter.
3. Specify a second pattern that matches any other alphabetic character and increments a consonant counter; let all remaining characters (digits, spaces, punctuation) be ignored.
4. In main(), call yylex() to run the generated lexical analyzer over the standard input, applying the above rules to every character read.
5. Once input ends, print the total number of vowels and consonants counted.

PROGRAM:

```
%{  
  
#include <stdio.h>  
  
int vowels = 0, consonants = 0;  
  
%}  
  
%%  
  
[aeiouAEIOU] { vowels++; }  
  
[a-zA-Z] { consonants++; }
```

```
. ;

\n ;

%%

int yywrap() {

return 1;

}

int main() {

printf("Enter a string (press Ctrl+D to end input):\n");

yylex();

printf("\nNumber of vowels = %d\n", vowels);

printf("Number of consonants = %d\n", consonants);

return 0;

}
```

EXPECTED OUTPUT:

Sample run for the input string "Hello World 123":

```
$ lex vowel.l
```

```
$ cc lex.yy.c -o vowel -ll
```

```
$ ./vowel
```

Enter a string (press Ctrl+D to end input):

Hello World 123

^D

Number of vowels = 3

Number of consonants = 7

MODEL VIVA QUESTIONS:

1. What is the role of the Lex tool in the design of a compiler, and what does it translate a .l file into?
2. What is the purpose of the yywrap() function, and what does its return value signify?
3. How does Lex decide which rule to apply when more than one pattern matches the input at a given position?

RESULT:

A Lex program was written to count the vowels and consonants in a given input string, and it was verified that the program correctly reports the total number of each for sample input strings.

EXP NO: 6

RECOGNIZE A VALID ARITHMETIC EXPRESSION USING YACC

AIM:

To generate a YACC specification that recognizes a valid arithmetic expression containing the operators $+$, $-$, $*$, $/$ and parentheses.

LEARNING OUTCOMES:

Upon completion of this experiment, the student will be able to:

1. Understand the structure of a YACC specification file.
2. Write grammar rules for arithmetic expressions.
3. Use precedence and associativity declarations in YACC.
4. Generate a parser using YACC and compile it with Lex.
5. Recognize valid arithmetic expressions using a parser.

DESCRIPTION:

YACC (Yet Another Compiler Compiler) is a parser generator used to construct syntax analyzers from context-free grammar specifications. It works together with Lex, where Lex performs lexical analysis and YACC performs syntax analysis. In this experiment, YACC checks whether an arithmetic expression containing identifiers, numbers, operators ($+$, $-$, $*$, $/$), and parentheses follows the grammar rules. If the expression is syntactically correct, the parser displays a success message; otherwise, it reports an error.

ALGORITHM:

1. Start.
2. Define tokens for identifiers and arithmetic operators.
3. Specify operator precedence and associativity.
4. Define grammar rules for arithmetic expressions.
5. Generate the parser using YACC.

6. Generate the lexical analyzer using Lex.
7. Compile both generated files.
8. Execute the program and enter an arithmetic expression.
9. Display whether the expression is valid or invalid.
10. Stop.

PROGRAM:

YACC Program (expr.y)

```
%{
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void yyerror(char *s);
```

```
int yylex();
```

```
%}
```

```
%token ID
```

```
%left '+' '-'
```

```
%left '*' '/'
```

```
%%
```

```
S : E { printf("Valid Arithmetic Expression\n"); }
```

```
;
```

```
E : E '+' E
```

```
  | E '-' E
```

```
  | E '*' E
```

```
  | E '/' E
```

```
| '(' E ')'  
  
| ID  
  
;  
  
%%  
  
int main()  
{  
  
    printf("Enter an arithmetic expression:\n");  
  
    yyparse();  
  
    return 0;  
}  
  
void yyerror(char *s)  
{  
  
    printf("Invalid Arithmetic Expression\n");  
}  
  
LEX Program (expr.l)  
  
%{  
  
#include "y.tab.h"  
  
%}  
  
%%  
  
[a-zA-Z][a-zA-Z0-9]*    { return ID; }  
  
[0-9]+                { return ID; }  
  
[\t ]                  ;
```

```
\n                return 0;\n\n                return yytext[0];\n\n%%\n\nint yywrap()\n{\n    return 1;\n}
```

PROCEDURE:

1. Type the YACC program and save it as expr.y.
2. Type the Lex program and save it as expr.l.
3. Generate the parser.

```
yacc -d expr.y
```

4. Generate the lexical analyzer.

```
flex expr.l
```

5. Compile the generated files.

```
gcc y.tab.c lex.yy.c -o expr -lfl
```

6. Execute the program.

```
./expr
```

7. Enter an arithmetic expression.
8. Observe whether the expression is valid or invalid.

SAMPLE INPUT:

```
a+b*(c-d)
```

SAMPLE OUTPUT:

Enter an arithmetic expression:

a+b*(c-d)

Valid Arithmetic Expression

MODEL VIVA QUESTIONS:

1. What is YACC?
2. What is the role of Lex in compiler design?
3. What is a parser?
4. What is context-free grammar?
5. What is operator precedence?

RESULT:

A YACC specification to recognize valid arithmetic expressions using operators $+$, $-$, $*$, $/$ and parentheses was implemented successfully, and the output was verified.

EXP NO: 7

RECOGNIZE A VALID IDENTIFIER USING YACC

AIM:

To generate a YACC specification that recognizes a valid identifier starting with a letter followed by any number of letters or digits.

LEARNING OUTCOMES:

Upon completion of this experiment, the student will be able to:

1. Understand the role of YACC in syntax analysis.
2. Write grammar rules to recognize valid identifiers.
3. Use Lex and YACC together to build a parser.
4. Generate and compile a parser using YACC and Flex.
5. Differentiate between valid and invalid identifiers.

DESCRIPTION:

YACC (Yet Another Compiler Compiler) is a parser generator that creates syntax analyzers from grammar specifications. In this experiment, Lex identifies an identifier token using regular expressions, while YACC checks whether the token satisfies the grammar rule for a valid identifier. A valid identifier begins with a letter and may be followed by any number of letters or digits. If the input satisfies the grammar, the parser accepts it; otherwise, it reports an error.

ALGORITHM:

1. Start.
2. Define a token for identifiers.
3. Write grammar rules to recognize an identifier.
4. Generate the parser using YACC.
5. Generate the lexical analyzer using Lex.

6. Compile the generated files.
7. Execute the program.
8. Enter an identifier.
9. Display whether the identifier is valid or invalid.
10. Stop.

PROGRAM:

YACC Program (identifier.y)

```
%{  
  
#include<stdio.h>  
  
#include<stdlib.h>  
  
void yyerror(char *s);  
  
int yylex();  
  
%}  
  
%token ID  
  
%%  
  
S : ID  
  
  {  
    printf("Valid Identifier\n");  
  }  
  
;  
  
%%
```

```
int main()

{

    printf("Enter an Identifier:\n");

    yyparse();

    return 0;

}
```

```
void yyerror(char *s)

{

    printf("Invalid Identifier\n");

}
```

LEX Program (identifier.l)

```
%{

#include "y.tab.h"

%}

%%

[a-zA-Z][a-zA-Z0-9]*    { return ID; }

[ \t]                  ;

\n                      return 0;

.                      return yytext[0];

%%

int yywrap()
```

```
{  
  
    return 1;  
  
}
```

PROCEDURE:

1. Type the YACC program and save it as identifier.y.
2. Type the Lex program and save it as identifier.l.
3. Generate the parser.

```
yacc -d identifier.y
```

4. Generate the lexical analyzer.

```
flex identifier.l
```

5. Compile the generated files.

```
gcc y.tab.c lex.yy.c -o identifier -lfl
```

6. Execute the program.

```
./identifier
```

7. Enter an identifier.

8. Observe whether the identifier is valid or invalid.

SAMPLE INPUT:

```
student123
```

SAMPLE OUTPUT:

```
Enter an Identifier:
```

```
student123
```

```
Valid Identifier
```

MODEL VIVA QUESTIONS:

1. What is an identifier?
2. What are the rules for naming identifiers in C?
3. What is YACC?
4. What is the purpose of Lex?
5. What is the role of yyparse()?

RESULT:

A YACC specification to recognize a valid identifier starting with a letter followed by any number of letters or digits was implemented successfully, and the output was verified.

EXP NO : 8

IMPLEMENTATION OF A CALCULATOR USING LEX AND YACC

AIM:

To implement a simple calculator using Lex and YACC that performs arithmetic operations such as addition, subtraction, multiplication, and division on integer operands.

LEARNING OUTCOMES:

Upon completion of this experiment, the student will be able to:

1. Understand the working of Lex and YACC together.
2. Develop a lexical analyzer using Lex.
3. Design grammar rules using YACC.
4. Implement arithmetic operations using parser actions.
5. Generate and execute a calculator using Lex and YACC.

DESCRIPTION:

Lex and YACC are compiler construction tools used together to build language processors. Lex performs lexical analysis by identifying tokens such as numbers and arithmetic operators, while YACC performs syntax analysis by checking whether the input follows the specified grammar rules. In this experiment, Lex recognizes numbers and operators, whereas YACC evaluates arithmetic expressions according to operator precedence and associativity. The calculator supports addition (+), subtraction (-), multiplication (*), division (/), and parentheses. After evaluating the expression, the result is displayed to the user.

ALGORITHM:

1. Start.
2. Define tokens for numbers and arithmetic operators.
3. Specify operator precedence and associativity in YACC.
4. Write grammar rules for arithmetic expressions.

5. Generate the parser using YACC.
6. Generate the lexical analyzer using Lex.
7. Compile the generated files.
8. Execute the program.
9. Enter an arithmetic expression.
10. Evaluate and display the result.
11. Stop.

PROGRAM:**YACC Program (calc.y)**

```
%{  
  
#include<stdio.h>  
  
#include<stdlib.h>  
  
int yylex();  
  
void yyerror(char *s);  
  
%}  
  
%token NUM  
  
%left '+' '-'  
  
%left '*' '/'  
  
%%  
  
input:  
    input line  
    |  
    ;  
  
line:
```

```
\n'

| exp '\n' { printf("Result = %d\n", $1); }

;

exp:

    NUM

    | exp '+' exp { $$ = $1 + $3; }

    | exp '-' exp { $$ = $1 - $3; }

    | exp '*' exp { $$ = $1 * $3; }

    | exp '/' exp

    {

        if($3==0)

        {

            printf("Division by Zero Error\n");

            $$=0;

        }

        else

            $$=$1/$3;

    }

    | '(' exp ')' { $$ = $2; }

;

%%

int main()
```

```
{ printf("Enter Arithmetic Expression:\n");

    yyparse();

    return 0;

}
```

```
void yyerror(char *s)
```

```
{ printf("Invalid Expression\n");

}
```

```
LEX Program (calc.l)
```

```
%{
```

```
#include "y.tab.h"
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
%}
```

```
%%
```

```
[0-9]+  {

        yylval=atoi(yytext);

        return NUM;

}
```

```
[\t]    ;
```

```
\n      return '\n';
```

```
.       return yytext[0];
```

```
%%
```

```
int yywrap()
```

```
{
```

```
    return 1;
```

```
}
```

PROCEDURE:

1. Type the YACC program and save it as calc.y.
2. Type the Lex program and save it as calc.l.
3. Open the terminal.
4. Generate the parser.

```
yacc -d calc.y
```

5. Generate the lexical analyzer.

```
flex calc.l
```

6. Compile the generated files.

```
gcc y.tab.c lex.yy.c -o calc -lfl
```

7. Execute the program.

```
./calc
```

8. Enter an arithmetic expression.

9. Observe the evaluated result.

SAMPLE INPUT:

```
25+15*2
```

SAMPLE OUTPUT:

```
Enter Arithmetic Expression:
```

25+15*2

Result = 55

MODEL VIVA QUESTIONS:

1. What is Lex?
2. What is YACC?
3. What is the difference between Lex and YACC?
4. What is a parser?
5. What is a lexical analyzer?

RESULT:

A calculator capable of performing arithmetic operations such as addition, subtraction, multiplication, and division was implemented successfully using Lex and YACC, and the output was verified.

EXP NO: 9

CONVERSION OF BNF RULES TO YACC FORM AND GENERATION OF ABSTRACT SYNTAX TREES

AIM:

To convert the given BNF (Backus-Naur Form) grammar rules for arithmetic expressions into YACC (Yet Another Compiler Compiler) specification form, and to write a LEX/YACC program that parses an input expression and generates its Abstract Syntax Tree (AST).

ALGORITHM:

1. Identify the BNF grammar rules of the language construct to be parsed (arithmetic expressions with +, -, *, / and parentheses).
2. Convert each BNF production into the equivalent YACC production, replacing terminals with tokens declared through %token and non-terminals with %type declarations.
3. Design the lexical specification (.l file) to recognize numbers, operators and parentheses, and return the corresponding tokens to the parser.
4. Define a structure "node" with fields for the token value and pointers to the left and right child nodes, to represent a node of the AST.
5. Write a function mknode() that dynamically allocates a new AST node, stores the token, and links the left and right sub-trees.
6. Embed semantic actions inside the YACC grammar rules so that, as each production is reduced, mknode() is called to combine the child sub-trees into a new parent node, building the AST bottom-up.
7. Write a function printtree() that performs a pre-order traversal of the AST and displays it with indentation proportional to depth.
8. Generate the lexical analyzer using LEX and the parser using YACC, compile both together with the C compiler, and link with the LEX/YACC libraries.
9. Execute the program with a sample arithmetic expression as input and verify that the printed AST correctly reflects the operator precedence and associativity of the grammar.

BNF GRAMMAR:

<expr> ::= <expr> + <term>

| <expr> - <term>

| <term>

<term> ::= <term> * <factor>

| <term> / <factor>

| <factor>

<factor> ::= (<expr>)

| id

| number

PROGRAM:

Lexical Specification File: expr.l

%{

#include "y.tab.h"

%}

%%

[0-9]+(\.[0-9]+)? { yylval.dval = atof(yytext); return NUMBER; }

[\t]+ ; /* skip whitespace */

\n return 0;

"+" return PLUS;

"-" return MINUS;

"*" return MUL;

"/" return DIV;

```
(" return LP;

)" return RP;

. return yytext[0];

%%

int yywrap() { return 1; }

YACC Specification File: expr.y

%{

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

typedef struct node {

char *token;

struct node *left;

struct node *right;

} node;

node *mknode(char *token, node *left, node *right) {

node *newnode = (node *) malloc(sizeof(node));

newnode->token = strdup(token);

newnode->left = left;

newnode->right = right;

return newnode;

}
```

```
void printtree(node *tree, int depth) {

int i;

if (tree == NULL) return;

for (i = 0; i < depth; i++) printf(" ");

printf("%s\n", tree->token);

printtree(tree->left, depth + 1);

printtree(tree->right, depth + 1);

}

int yylex();

void yyerror(char *s);

node *root;

%}

%union {

struct node *nptr;

double dval;

}

%token <dval> NUMBER

%token PLUS MINUS MUL DIV LP RP

%type <nptr> expr term factor

%%

program:

expr '\n'
```

```
{  
  
root = $1;  
  
printf("\nAbstract Syntax Tree:\n");  
  
printtree(root, 0);  
  
}  
  
;  
  
expr:  
  
expr PLUS term { $$ = mknode("+", $1, $3); }  
  
| expr MINUS term { $$ = mknode("-", $1, $3); }  
  
| term { $$ = $1; }  
  
;  
  
term:  
  
term MUL factor { $$ = mknode("*", $1, $3); }  
  
| term DIV factor { $$ = mknode("/", $1, $3); }  
  
| factor { $$ = $1; }  
  
;  
  
factor:  
  
LP expr RP { $$ = $2; }  
  
| NUMBER  
  
{  
  
char buf[32];  
  
sprintf(buf, "%g", $1);
```

```
$$ = mknode(buf, NULL, NULL);

}

;

%%

void yyerror(char *s) {

printf("Error: %s\n", s);

}

int main() {

printf("Enter an arithmetic expression: ");

yyvsparse();

return 0;

}
```

Compilation and Execution Commands

```
$ lex expr.l

$ yacc -d expr.y

$ cc lex.yy.c y.tab.c -o parser -ll

$ ./parser
```

EXPECTED OUTPUT:

Enter an arithmetic expression: 3 + 4 * (2 - 1)

Abstract Syntax Tree:

+

3

*

4

-

2

1

MODEL VIVA QUESTIONS:

1. What is the difference between BNF notation and YACC grammar notation?
2. What is the purpose of the %union declaration in a YACC specification?
3. How does YACC resolve ambiguity and enforce operator precedence and associativity?
4. What is an Abstract Syntax Tree, and how does it differ from a parse tree?
5. Why is the AST constructed bottom-up during a YACC (LALR) parse?

RESULT:

The given BNF grammar for arithmetic expressions was successfully converted into YACC form, and a LEX-YACC program was implemented that parses an input expression and constructs and displays its Abstract Syntax Tree.

EXP NO: 10

COMPUTATION OF EPSILON-CLOSURE OF ALL STATES OF A GIVEN NFA
WITH EPSILON TRANSITIONS

AIM:

To write a C program to find the epsilon-closure (ϵ -closure) of every state of a given Non-deterministic Finite Automaton (NFA) that contains epsilon (ϵ) transitions.

ALGORITHM:

1. Read the number of states n and the number of input symbols (excluding epsilon) of the NFA, along with the input alphabet.
2. Read the transition table of the NFA, including the epsilon transitions, and store it using an adjacency-list representation indexed by state and symbol (an extra symbol slot is reserved for epsilon).
3. For every state q of the NFA, initialize an empty visited set and mark q itself as visited (q is always a member of its own epsilon-closure).
4. Perform a Depth First Search (DFS) starting from q , moving only along epsilon transitions, and mark every state reached as visited.
5. Continue the DFS recursively until no new state can be reached through epsilon transitions from the states already visited.
6. The set of all visited states after the DFS terminates constitutes ϵ -closure(q).
7. Repeat steps 3 to 6 for every state of the NFA and display the epsilon-closure of each state.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#define MAXSTATES 20

int n; /* number of states */

int numSymbols; /* number of input symbols, excluding epsilon */
```

```
char symbols[MAXSTATES]; /* input alphabet */

int trans[MAXSTATES][MAXSTATES + 1][MAXSTATES]; /* trans[state][symbolIndex][k] */

int transCount[MAXSTATES][MAXSTATES + 1];

int visited[MAXSTATES];

/* epsilon is stored at symbol index = numSymbols */

void dfsEpsilon(int state, int epsIndex) {

    int k, next;

    if (visited[state]) return;

    visited[state] = 1;

    for (k = 0; k < transCount[state][epsIndex]; k++) {

        next = trans[state][epsIndex][k];

        dfsEpsilon(next, epsIndex);

    }

}

int main()

{

    int i, j, k, dest, cnt, epsIndex;

    printf("Enter number of states: ");

    scanf("%d", &n);

    printf("Enter number of input symbols (excluding epsilon): ");

    scanf("%d", &numSymbols);

    for (i = 0; i < numSymbols; i++) {
```

```
printf("Enter symbol %d: ", i + 1);

scanf(" %c", &symbols[i]);

}

epsIndex = numSymbols; /* last column reserved for epsilon */

printf("\nEnter transitions (enter -1 to end the list for each state/symbol):\n");

for (i = 0; i < n; i++) {

    for (j = 0; j <= numSymbols; j++) {

        if (j < numSymbols)

            printf("From state q%d on symbol %c: ", i, symbols[j]);

        else

            printf("From state q%d on epsilon: ", i);

        cnt = 0;

        while (1) {

            scanf("%d", &dest);

            if (dest == -1) break;

            trans[i][j][cnt++] = dest;

        }

        transCount[i][j] = cnt;

    }

}

printf("\nEpsilon-Closures:\n");

for (i = 0; i < n; i++) {
```

```
for (k = 0; k < MAXSTATES; k++) visited[k] = 0;

dfsEpsilon(i, epsIndex);

printf("e-closure(q%d) = { ", i);

for (k = 0; k < n; k++)

if (visited[k]) printf("q%d ", k);

printf("}\n");

}

return 0;

}
```

Compilation and Execution Commands

```
$ cc eclosure.c -o eclosure
```

```
$ ./eclosure
```

EXPECTED OUTPUT:

Sample NFA: states q0, q1, q2 ; symbols a, b ; epsilon transitions $q_0 \rightarrow q_1$ and $q_1 \rightarrow q_2$.

Enter number of states: 3

Enter number of input symbols (excluding epsilon): 2

Enter symbol 1: a

Enter symbol 2: b

Enter transitions (enter -1 to end the list for each state/symbol):

From state q0 on symbol a: -1

From state q0 on symbol b: -1

From state q0 on epsilon: 1 -1

From state q1 on symbol a: -1

From state q1 on symbol b: -1

From state q1 on epsilon: 2 -1

From state q2 on symbol a: -1

From state q2 on symbol b: -1

From state q2 on epsilon: -1

Epsilon-Closures:

$e\text{-closure}(q_0) = \{ q_0 \ q_1 \ q_2 \}$

$e\text{-closure}(q_1) = \{ q_1 \ q_2 \}$

$e\text{-closure}(q_2) = \{ q_2 \}$

MODEL VIVA QUESTIONS:

1. What is an epsilon (ϵ) transition in an NFA?
2. Why does every state always belong to its own epsilon-closure?
3. How is the epsilon-closure of a set of states computed, as opposed to a single state?
4. What role does epsilon-closure play in the subset construction algorithm used for NFA to DFA conversion?
5. Can a graph traversal technique other than DFS be used to compute epsilon-closure? Justify.

RESULT:

A C program was implemented to compute the epsilon-closure of every state of a given NFA containing epsilon transitions, and the program was executed and verified successfully for a sample NFA.

EXP NO: 11

CONVERSION OF NFA WITH EPSILON TRANSITIONS TO NFA WITHOUT EPSILON TRANSITIONS

AIM:

To write a C program that eliminates epsilon (ϵ) transitions from a given NFA and constructs an equivalent NFA without epsilon transitions.

ALGORITHM:

1. Read the NFA — states, alphabet, epsilon transitions, normal transitions and final states.
2. Compute ϵ -closure(q) for every state q.
3. For each state q and symbol a, set new-move(q, a) = ϵ -closure($\cup$ move(p, a)), for every p in ϵ -closure(q).
4. Mark q as a final state in the new NFA if ϵ -closure(q) contains any original final state.
5. Discard all epsilon transitions and output the new transition table.

PROGRAM:

```
#include <stdio.h>

#define MAX 20

int n, m;

char sym[MAX];

int epsAdj[MAX][MAX], epsCount[MAX];

int trans[MAX][MAX][MAX], transCount[MAX][MAX]; /* trans[state][symbolIdx][k] */

int isFinal[MAX], visited[MAX];

void dfsEps(int s) {

    int k;
```

```
if (visited[s]) return;

visited[s] = 1;

for (k = 0; k < epsCount[s]; k++) dfsEps(epsAdj[s][k]);

}

int main() {

int i, j, k, p, q, d, dest, cnt;

int ecl[MAX][MAX], eclCnt[MAX];

printf("Enter number of states: "); scanf("%d", &n);

printf("Enter number of symbols (excluding epsilon): "); scanf("%d", &m);

for (i = 0; i < m; i++) { printf("Symbol %d: ", i + 1); scanf(" %c", &sym[i]); }

for (i = 0; i < n; i++) {

printf("Epsilon moves of q%d (-1 to stop): ", i);

cnt = 0;

while (1) { scanf("%d", &dest); if (dest == -1) break; epsAdj[i][cnt++] = dest; }

epsCount[i] = cnt;

}

for (i = 0; i < n; i++)

for (j = 0; j < m; j++) {

printf("q%d on %c (-1 to stop): ", i, sym[j]);

cnt = 0;

while (1) { scanf("%d", &dest); if (dest == -1) break; trans[i][j][cnt++] = dest; }

transCount[i][j] = cnt;
```

```
}

printf("Final states (-1 to stop): ");

while (1) { scanf("%d", &dest); if (dest == -1) break; isFinal[dest] = 1; }

for (i = 0; i < n; i++) {

for (k = 0; k < MAX; k++) visited[k] = 0;

dfsEps(i);

cnt = 0;

for (k = 0; k < n; k++) if (visited[k]) ecl[i][cnt++] = k;

eclCnt[i] = cnt;

}

printf("\nEquivalent NFA without epsilon transitions:\n");

for (i = 0; i < n; i++) {

for (j = 0; j < m; j++) {

int used[MAX] = {0}, newset[MAX], nc = 0;

for (p = 0; p < eclCnt[i]; p++) {

int st = ecl[i][p];

for (k = 0; k < transCount[st][j]; k++) {

d = trans[st][j][k];

for (q = 0; q < eclCnt[d]; q++) {

int r = ecl[d][q];

if (!used[r]) { used[r] = 1; newset[nc++] = r; }

}

}
```

```
}  
  
}  
  
printf("delta(q%d, %c) = { ", i, sym[j]);  
  
for (k = 0; k < nc; k++) printf("q%d ", newset[k]);  
  
printf("}\n");  
  
}  
  
}  
  
printf("New final states: ");  
  
for (i = 0; i < n; i++) {  
  
    int isNewFinal = 0;  
  
    for (k = 0; k < eclCnt[i]; k++) if (isFinal[ecl[i][k]]) isNewFinal = 1;  
  
    if (isNewFinal) printf("q%d ", i);  
  
}  
  
printf("\n");  
  
return 0;  
  
}
```

EXPECTED OUTPUT:

Sample NFA: states q_0, q_1, q_2 ; symbol $a ; \epsilon$: $q_0 \rightarrow q_1, q_1 \rightarrow q_2$; $\text{move}(q_2, a) = q_2$; q_2 is final.

Equivalent NFA without epsilon transitions:

$\text{delta}(q_0, a) = \{ q_2 \}$

$\text{delta}(q_1, a) = \{ q_2 \}$

$\text{delta}(q_2, a) = \{ q_2 \}$

New final states: q0 q1 q2

MODEL VIVA QUESTIONS:

1. Why is epsilon-closure required to eliminate ϵ -transitions from an NFA?
2. Does eliminating ϵ -transitions change the language accepted by the NFA?
3. How are the final states of the new NFA determined?

RESULT:

A C program was implemented to eliminate epsilon transitions from a given NFA, and the equivalent NFA without epsilon transitions was obtained and verified.

EXP NO: 12

CONVERSION OF NFA TO DFA (SUBSET CONSTRUCTION)

AIM:

To write a C program that converts a given Non-deterministic Finite Automaton (NFA) into an equivalent Deterministic Finite Automaton (DFA) using the subset construction method.

ALGORITHM:

1. Read the NFA — states, alphabet, transition function and final states.
2. Represent each DFA state as a subset (bitmask) of NFA states; the DFA start state is { NFA start state }.
3. For each unmarked DFA state and each input symbol, compute move = $\cup$ of NFA transitions of all states in that subset; if the resulting subset is new, add it to the DFA.
4. Repeat step 3 until no new subsets are produced.
5. Mark a DFA state as final if its subset contains any NFA final state, then output the DFA transition table.

PROGRAM:

```
#include <stdio.h>

#define MAXSTATES 10

#define MAXSYM 5

#define MAXDFA (1 << MAXSTATES)

int n, m;

char sym[MAXSYM];

int trans[MAXSTATES][MAXSYM][MAXSTATES], transCount[MAXSTATES][MAXSYM];

int isFinal[MAXSTATES];
```

```
int visited[MAXDFA], queueArr[MAXDFA], front = 0, rear = 0;

int move(int subset, int j) {

int p, k, result = 0;

for (p = 0; p < n; p++)

if (subset & (1 << p))

for (k = 0; k < transCount[p][j]; k++)

result |= (1 << trans[p][j][k]);

return result;

}

void printSubset(int s) {

int p, first = 1;

printf("{");

for (p = 0; p < n; p++)

if (s & (1 << p)) { printf("%sq%d", first ? "" : ", ", p); first = 0; }

printf("}");

}

int main() {

int i, j, dest, cnt, start, cur, nxt;

printf("Enter number of NFA states: "); scanf("%d", &n);

printf("Enter number of symbols: "); scanf("%d", &m);

for (i = 0; i < m; i++) { printf("Symbol %d: ", i + 1); scanf(" %c", &sym[i]); }

for (i = 0; i < n; i++)
```

```
for (j = 0; j < m; j++) {

printf("q%d on %c (-1 to stop): ", i, sym[j]);

cnt = 0;

while (1) { scanf("%d", &dest); if (dest == -1) break; trans[i][j][cnt++] = dest; }

transCount[i][j] = cnt;

}

printf("Start state: "); scanf("%d", &start);

printf("Final states (-1 to stop): ");

while (1) { scanf("%d", &dest); if (dest == -1) break; isFinal[dest] = 1; }

int s0 = 1 << start;

visited[s0] = 1; queueArr[rear++] = s0;

printf("\nDFA Transition Table:\n");

while (front < rear) {

cur = queueArr[front++];

for (j = 0; j < m; j++) {

nxt = move(cur, j);

if (nxt == 0) continue;

printSubset(cur); printf(" --%c--> ", sym[j]); printSubset(nxt); printf("\n");

if (!visited[nxt]) { visited[nxt] = 1; queueArr[rear++] = nxt; }

}

}

printf("\nDFA Final States: ");
```

```
for (i = 0; i < rear; i++) {  
  
    cur = queueArr[i];  
  
    for (j = 0; j < n; j++)  
  
        if ((cur & (1 << j)) && isFinal[j]) { printSubset(cur); printf(" "); break; }  
  
    printf("\n");  
  
    return 0;  
  
}
```

EXPECTED OUTPUT:

Sample NFA: states q0, q1 ; symbol a ; move(q0,a) = {q0,q1} ; move(q1,a) = {} ; start = q0 ; final = q1.

DFA Transition Table:

{q0} --a--> {q0,q1}

{q0,q1} --a--> {q0,q1}

DFA Final States: {q0,q1}

MODEL VIVA QUESTIONS:

1. What is the subset construction algorithm used for?
2. Why can the number of DFA states be exponential in the number of NFA states?
3. How is a DFA state marked as final in subset construction?

RESULT:

A C program was implemented to convert a given NFA into an equivalent DFA using the subset construction method, and the resulting DFA transition table was obtained and verified.

EXP NO: 13

MINIMIZATION OF A GIVEN DFA

AIM:

To write a C program to minimize a given Deterministic Finite Automaton (DFA) by merging equivalent states, using the partition-refinement (equivalence class) method.

ALGORITHM:

1. Read the DFA — states, alphabet, transition function, start state and final states.
2. Partition the states into two initial groups: final states and non-final states.
3. For each group, split it further into subgroups whenever two states in the same group go to different groups on some input symbol.
4. Repeat step 3, refining the partition, until it stabilizes (no group can be split any further).
5. Treat every remaining group as one state of the minimized DFA, and construct its transition table using the transitions of any representative state of each group.
6. A group is a final state of the minimized DFA if it contains at least one original final state.

PROGRAM:

```
#include <stdio.h>

#define MAXSTATES 20

#define MAXSYM 10

int n, m;

char sym[MAXSYM];

int trans[MAXSTATES][MAXSYM]; /* DFA: single destination per state/symbol */

int isFinal[MAXSTATES];

int group[MAXSTATES];

int main() {
```

```
int i, j, k, changed, nextId, d;

printf("Enter number of states: "); scanf("%d", &n);

printf("Enter number of symbols: "); scanf("%d", &m);

for (i = 0; i < m; i++) { printf("Symbol %d: ", i + 1); scanf(" %c", &sym[i]); }

for (i = 0; i < n; i++)

for (j = 0; j < m; j++) {

printf("delta(q%d, %c): ", i, sym[j]);

scanf("%d", &trans[i][j]);

}

printf("Final states (-1 to stop): ");

while (1) { scanf("%d", &d); if (d == -1) break; isFinal[d] = 1; }

for (i = 0; i < n; i++) group[i] = isFinal[i] ? 1 : 0;

do {

int assigned[MAXSTATES];

int newgroup[MAXSTATES];

for (i = 0; i < n; i++) assigned[i] = -1;

nextId = 0;

changed = 0;

for (i = 0; i < n; i++) {

if (assigned[i] != -1) continue;

assigned[i] = nextId;

for (k = i + 1; k < n; k++) {
```

```
if (assigned[k] != -1 || group[i] != group[k]) continue;

int same = 1;

for (j = 0; j < m; j++)

if (group[trans[i][j]] != group[trans[k][j]]) { same = 0; break; }

if (same) assigned[k] = nextId;

}

nextId++;

}

for (i = 0; i < n; i++) {

if (assigned[i] != group[i]) changed = 1;

newgroup[i] = assigned[i];

}

for (i = 0; i < n; i++) group[i] = newgroup[i];

} while (changed);

printf("\nEquivalent State Groups (Minimized States):\n");

{ int printed[MAXSTATES] = {0};

for (i = 0; i < n; i++) {

if (printed[group[i]]) continue;

printed[group[i]] = 1;

printf("G%d = { ", group[i]);

for (k = 0; k < n; k++) if (group[k] == group[i]) printf("q%d ", k);

printf("}\n");
```

```
}  
  
}  
  
printf("\nMinimized DFA Transition Table:\n");  
  
{ int shown[MAXSTATES] = {0};  
  
for (i = 0; i < n; i++) {  
  
if (shown[group[i]]) continue;  
  
shown[group[i]] = 1;  
  
for (j = 0; j < m; j++)  
  
printf("delta(G%d, %c) = G%d\n", group[i], sym[j], group[trans[i][j]]);  
  
}  
  
}  
  
printf("\nMinimized DFA Final States: ");  
  
{ int shownf[MAXSTATES] = {0};  
  
for (i = 0; i < n; i++)  
  
if (isFinal[i] && !shownf[group[i]]) { shownf[group[i]] = 1; printf("G%d ", group[i]); }  
  
}  
  
printf("\n");  
  
return 0;  
  
}
```

EXPECTED OUTPUT:

Sample DFA: states A(0),B(1),C(2),D(3),E(4),F(5) ; symbols a,b ; start A ; final = {C,D,E}.

delta(A,a)=B delta(A,b)=C

$\delta(B,a)=A \quad \delta(B,b)=D$ $\delta(C,a)=E \quad \delta(C,b)=F$ $\delta(D,a)=E \quad \delta(D,b)=F$ $\delta(E,a)=E \quad \delta(E,b)=F$ $\delta(F,a)=F \quad \delta(F,b)=F$

Equivalent State Groups (Minimized States):

 $G_0 = \{ q_0, q_1 \}$ $G_1 = \{ q_2, q_3, q_4 \}$ $G_2 = \{ q_5 \}$

Minimized DFA Transition Table:

 $\delta(G_0, a) = G_0$ $\delta(G_0, b) = G_1$ $\delta(G_1, a) = G_1$ $\delta(G_1, b) = G_2$ $\delta(G_2, a) = G_2$ $\delta(G_2, b) = G_2$

Minimized DFA Final States: G_1

MODEL VIVA QUESTIONS:

1. What does it mean for two DFA states to be equivalent?
2. Why does the partition-refinement process always terminate?
3. How is the minimized DFA guaranteed to accept the same language as the original DFA?

RESULT:

A C program was implemented to minimize a given DFA using the partition-refinement method, and the minimized DFA transition table was obtained and verified.

EXP NO: 14

COMPUTATION OF FIRST AND FOLLOW OF A GIVEN GRAMMAR

AIM:

To write a C program to compute the FIRST and FOLLOW sets of every non-terminal of a given context-free grammar.

ALGORITHM:

1. Read the grammar productions (in the form LHS = RHS, one alternative per production; '#' denotes epsilon) and the start symbol.
2. FIRST of a terminal is the terminal itself. FIRST of a non-terminal A is computed from every production $A \rightarrow X_1X_2\dots X_k$: add $\text{FIRST}(X_1) - \{\epsilon\}$; if X_1 is nullable, add $\text{FIRST}(X_2) - \{\epsilon\}$ and so on; if every X_i is nullable, add ϵ to $\text{FIRST}(A)$.
3. Add '\$' (end marker) to FOLLOW of the start symbol.
4. For a production $A \rightarrow \alpha B \beta$: add $\text{FIRST}(\beta) - \{\epsilon\}$ to $\text{FOLLOW}(B)$; if β is empty or nullable, also add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.
5. Repeat the FIRST and FOLLOW computations for every non-terminal until no further symbols can be added to any set.
6. Display the FIRST and FOLLOW sets of all non-terminals.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#define MAX 20

char prodLHS[MAX]; /* left-hand-side non-terminal of production i */

char prodRHS[MAX][MAX]; /* right-hand-side string; '#' denotes epsilon */

int numProd;

char startSymbol;
```

```
char ntList[MAX]; int ntCount = 0;

char firstSet[26][MAX]; int firstCount[26] = {0};

char followSet[26][MAX]; int followCount[26] = {0};

int firstDone[26] = {0}, followDone[26] = {0};

int isTerminal(char c) { return !(c >= 'A' && c <= 'Z'); }

int inSet(char set[], int cnt, char c) {

int i; for (i = 0; i < cnt; i++) if (set[i] == c) return 1; return 0;

}

void addToFirst(int nt, char c) { if (!inSet(firstSet[nt], firstCount[nt], c)) firstSet[nt][firstCount[nt]++] = c;
}

void addToFollow(int nt, char c) { if (!inSet(followSet[nt], followCount[nt], c))
followSet[nt][followCount[nt]++] = c; }

void findFirst(char c) {

int nt = c - 'A', i, j, k, hasEps;

if (firstDone[nt]) return;

firstDone[nt] = 1;

for (i = 0; i < numProd; i++) {

if (prodLHS[i] != c) continue;

char *rhs = prodRHS[i];

if (rhs[0] == '#') { addToFirst(nt, '#'); continue; }

for (j = 0; rhs[j] != '\0'; j++) {

char sym = rhs[j];
```

```
if (isTerminal(sym)) { addToFirst(nt, sym); break; }

if (!firstDone[sym - 'A']) findFirst(sym);

hasEps = 0;

for (k = 0; k < firstCount[sym - 'A']; k++) {

if (firstSet[sym - 'A'][k] == '#') hasEps = 1;

else addToFirst(nt, firstSet[sym - 'A'][k]);

}

if (!hasEps) break;

if (rhs[j + 1] == '\0') addToFirst(nt, '#');

}

}

}

void findFollow(char c) {

int nt = c - 'A', i, j, k, m, hasEps, len;

if (followDone[nt]) return;

followDone[nt] = 1;

if (c == startSymbol) addToFollow(nt, '$');

for (i = 0; i < numProd; i++) {

char *rhs = prodRHS[i];

len = strlen(rhs);

for (j = 0; j < len; j++) {

if (rhs[j] != c) continue;
```

```
if (j == len - 1) { /* c is the last symbol of the RHS */

if (prodLHS[i] != c) {

findFollow(prodLHS[i]);

for (k = 0; k < followCount[prodLHS[i]-'A']; k++)

addToFollow(nt, followSet[prodLHS[i]-'A'][k]);

}

} else {

char next = rhs[j + 1];

if (isTerminal(next)) { addToFollow(nt, next); continue; }

if (!firstDone[next - 'A']) findFirst(next);

hasEps = 0;

for (m = 0; m < firstCount[next - 'A']; m++) {

if (firstSet[next - 'A'][m] == '#') hasEps = 1;

else addToFollow(nt, firstSet[next - 'A'][m]);

}

if (hasEps && j + 1 == len - 1 && prodLHS[i] != c) {

findFollow(prodLHS[i]);

for (k = 0; k < followCount[prodLHS[i]-'A']; k++)

addToFollow(nt, followSet[prodLHS[i]-'A'][k]);

}

}

}
```

```
}  
  
}  
  
int main() {  
  
    int i, j;  
  
    char buf[MAX];  
  
    printf("Enter number of productions: "); scanf("%d", &numProd);  
  
    printf("Enter each production as LHS=RHS ('#' for epsilon):\n");  
  
    for (i = 0; i < numProd; i++) {  
  
        scanf("%s", buf);  
  
        prodLHS[i] = buf[0];  
  
        strcpy(prodRHS[i], buf + 2);  
  
        if (!inSet(ntList, ntCount, buf[0])) ntList[ntCount++] = buf[0];  
  
    }  
  
    printf("Enter start symbol: "); scanf(" %c", &startSymbol);  
  
    for (i = 0; i < ntCount; i++) findFirst(ntList[i]);  
  
    for (i = 0; i < ntCount; i++) findFollow(ntList[i]);  
  
    printf("\nFIRST and FOLLOW sets:\n");  
  
    for (i = 0; i < ntCount; i++) {  
  
        int nt = ntList[i] - 'A';  
  
        printf("FIRST(%c) = { ", ntList[i]);  
  
        for (j = 0; j < firstCount[nt]; j++) printf("%c ", firstSet[nt][j]);  
  
        printf("}\n");  
    }  
}
```

```
printf("FOLLOW(%c) = { ", ntList[i]);  
  
for (j = 0; j < followCount[nt]; j++) printf("%c ", followSet[nt][j]);  
  
printf("}\n");  
  
}  
  
return 0;  
  
}
```

EXPECTED OUTPUT:

Sample grammar (expression grammar with left recursion removed), start symbol E:

$E = TX$

$X = +TX$

$X = \#$

$T = FY$

$Y = *FY$

$Y = \#$

$F = (E)$

$F = i$

FIRST and FOLLOW sets:

$\text{FIRST}(E) = \{ (i \}$

$\text{FOLLOW}(E) = \{ \$) \}$

$\text{FIRST}(X) = \{ + \# \}$

$\text{FOLLOW}(X) = \{ \$) \}$

$\text{FIRST}(T) = \{ (i \}$

$\text{FOLLOW}(T) = \{ + \$) \}$

$\text{FIRST}(Y) = \{ * \# \}$

$\text{FOLLOW}(Y) = \{ + \$) \}$

$\text{FIRST}(F) = \{ (i \}$

$\text{FOLLOW}(F) = \{ * + \$) \}$

MODEL VIVA QUESTIONS:

1. Why is FIRST and FOLLOW computation needed in compiler design?
2. What does it mean for a non-terminal to be nullable?
3. How are FIRST and FOLLOW used while constructing a predictive (LL(1)) parsing table?

RESULT:

A C program was implemented to compute the FIRST and FOLLOW sets of all non-terminals of a given grammar, and the output was verified for a sample expression grammar.

EXP NO: 15

DESIGN AND IMPLEMENTATION OF A RECURSIVE DESCENT PARSER

AIM:

To design and implement a Recursive Descent Parser in C for a given grammar, and to verify whether a given input string is accepted by the grammar.

ALGORITHM:

1. Eliminate left recursion and perform left factoring on the given grammar, if necessary, so that it becomes suitable for LL(1) top-down parsing.
2. For every non-terminal of the grammar, write a corresponding C function that implements its productions: match terminals against the current input symbol, and call the functions of non-terminals recursively.
3. Maintain a global pointer to track the current lookahead symbol in the input string.
4. Write a match() function that advances the pointer if the current symbol equals the expected terminal, and reports a syntax error otherwise.
5. Begin parsing by invoking the function corresponding to the start symbol.
6. If the entire input is consumed (end marker '\$' reached) with no mismatch, report that the string is successfully parsed; otherwise report a syntax error.

GRAMMAR (LEFT RECURSION ELIMINATED):

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <string.h>

char input[50];

int pos = 0;

void E();

void Eprime();

void T();

void Tprime();

void F();

void match(char expected) {

if (input[pos] == expected) {

pos++;

} else {

printf("Syntax Error: expected '%c' but found '%c'\n", expected, input[pos]);

exit(1);

}

}

void E() {

printf("E -> T E\n");

T();

Eprime();

}
```

```
void Eprime() {  
  
    if (input[pos] == '+') {  
  
        printf("E' -> + T E\n");  
  
        match('+');  
  
        T();  
  
        Eprime();  
  
    } else {  
  
        printf("E' -> epsilon\n");  
  
    }  
  
}  
  
void T() {  
  
    printf("T -> F T\n");  
  
    F();  
  
    Tprime();  
  
}  
  
void Tprime() {  
  
    if (input[pos] == '*') {  
  
        printf("T' -> * F T\n");  
  
        match('*');  
  
        F();  
  
        Tprime();  
  
    } else {
```

```
printf("T' -> epsilon\n");

}

}

void F() {

if (input[pos] == '(') {

printf("F -> ( E )\n");

match('(');

E();

match(')');

} else if (input[pos] == 'i') {

printf("F -> id\n");

match('i');

} else {

printf("Syntax Error: unexpected symbol '%c'\n", input[pos]);

exit(1);

}

}

int main() {

printf("Enter input string (use 'i' for id, end with $): ");

scanf("%s", input);

E();

if (input[pos] == '$')
```

```
printf("\nInput string is successfully parsed by the Recursive Descent Parser.\n");  
  
else  
  
printf("\nSyntax Error: unexpected symbol '%c'\n", input[pos]);  
  
return 0;  
  
}
```

EXPECTED OUTPUT:

Sample run for the input string $i+i*i$ (represented as $i+i*i\$$, where 'i' denotes id):

Enter input string (use 'i' for id, end with \$): $i+i*i\$$

$E \rightarrow T E'$

$T \rightarrow F T'$

$F \rightarrow id$

$T' \rightarrow \epsilon$

$E' \rightarrow + T E'$

$T \rightarrow F T'$

$F \rightarrow id$

$T' \rightarrow * F T'$

$F \rightarrow id$

$T' \rightarrow \epsilon$

$E' \rightarrow \epsilon$

Input string is successfully parsed by the Recursive Descent Parser.

MODEL VIVA QUESTIONS:

1. Why must left recursion be eliminated before writing a recursive descent parser?

2. What is the role of the lookahead symbol in a recursive descent parser?
3. What is the relationship between a recursive descent parser and an LL(1) grammar?

RESULT:

A Recursive Descent Parser was designed and implemented in C for the given grammar, and it was verified that valid input strings are successfully parsed while invalid strings are correctly rejected.

EXP NO: 16

CONSTRUCTION OF A SHIFT-REDUCE PARSER FOR A GIVEN LANGUAGE

AIM:

To construct a Shift-Reduce Parser in C for a given expression grammar, and to verify whether a given input string is accepted, using a stack-based shift/reduce trace.

ALGORITHM:

1. Read the grammar productions and the input string, terminated by the end marker '\$'.
2. Initialize an empty stack and set the input pointer to the first symbol of the string.
3. At each step, check whether the top of the stack matches the right-hand side (the “handle”) of some production; if it does, reduce — pop the handle and push the corresponding left-hand-side non-terminal.
4. If no handle is found, shift the next input symbol from the input onto the stack and advance the input pointer.
5. Repeat steps 3 and 4 until the input is fully consumed and no further handle can be reduced.
6. If the stack finally contains only the start symbol, report the string as accepted; otherwise report it as rejected.

GRAMMAR:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow id$

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char *matchStr[] = { "i", "(E)", "E+E", "E*E" };

char *displayStr[] = { "E -> id", "E -> (E)", "E -> E+E", "E -> E*E" };

int numProds = 4;

int reduceStack(char *stk, char *action) {

    int i, len, slen = strlen(stk);

    for (i = 0; i < numProds; i++) {

        len = strlen(matchStr[i]);

        if (slen >= len && strcmp(stk + slen - len, matchStr[i]) == 0) {

            stk[slen - len] = 'E';

            stk[slen - len + 1] = '\0';

            sprintf(action, "Reduce (%s)", displayStr[i]);

            return 1;

        }

    }

    return 0;

}

int main() {

    char inp[30], stk[30] = "", action[30];

    int ip = 0;

    printf("Enter input string (use 'i' for id, end with $): ");

    scanf("%s", inp);

    printf("\n%-15s%-15s%-15s\n", "Stack", "Input", "Action");
```

```
while (1) {

printf("%-15s%-15s", stk, inp + ip);

if (reduceStack(stk, action)) {

printf("%s\n", action);

continue;

}

if (inp[ip] != '$') {

int len = strlen(stk);

stk[len] = inp[ip];

stk[len + 1] = '\0';

ip++;

printf("Shift\n");

continue;

}

printf("-\n");

break;

}

if (strcmp(stk, "E") == 0)

printf("\nAccepted: The input string is successfully parsed.\n");

else

printf("\nRejected: The input string is not valid.\n");

return 0;
```

```
}
```

EXPECTED OUTPUT:

Sample run for the input string $i+i*i$ (entered as $i+i*i\$$):

Stack Input Action

$i+i*i\$$ Shift

$i+i*i\$$ Reduce ($E \rightarrow id$)

$E+i*i\$$ Shift

$E+i*i\$$ Shift

$E+i*i\$$ Reduce ($E \rightarrow id$)

$E+E*i\$$ Reduce ($E \rightarrow E+E$)

$E*i\$$ Shift

$E*i\$$ Shift

$E*i\$$ Reduce ($E \rightarrow id$)

$E*i\$$ Reduce ($E \rightarrow E*i$)

$E\$$ -

Accepted: The input string is successfully parsed.

MODEL VIVA QUESTIONS:

1. What are the two basic actions performed by a shift-reduce parser?
2. What is meant by a “handle” in shift-reduce parsing?
3. What is a shift-reduce conflict, and when does it occur?

RESULT:

A Shift-Reduce Parser was constructed in C for the given expression grammar, and it was verified that valid input strings are accepted while invalid strings are rejected.

EXP NO: 17

IMPLEMENTATION OF CONSTANT PROPAGATION

AIM:

To implement Constant Propagation, a code-optimization technique, in C for a given set of three-address statements, and to verify which variables and expressions can be replaced by their compile-time constant values.

ALGORITHM:

1. Read the number of three-address statements and the statements themselves, each of the form `dest=operand`, `dest=op1 operator op2`, or `print var`.
2. Maintain a symbol table that records, for every variable, whether its value is currently known to be a constant and, if so, what that value is.
3. For a simple assignment `dest=operand`, look up the operand: if it is a literal or a variable already marked constant, mark `dest` as constant with that value; otherwise mark `dest` as non-constant.
4. For a binary statement `dest=op1 operator op2`, look up both operands. If both are known constants, evaluate the expression at compile time (constant folding) and mark `dest` as constant.
5. If only one operand is a known constant, substitute its value into the expression (partial constant propagation) but mark `dest` as non-constant, since the result still depends on a variable.
6. If neither operand is constant, leave the statement unchanged and mark `dest` as non-constant.
7. For a print statement, report whether the printed variable currently holds a known constant value, and print that value if so.
8. Repeat steps 3 to 7 for every statement, displaying the original statement, its optimized form, and a remark describing the action taken.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>
```

```
#include <stdlib.h>

#define MAX 20

char varName[MAX][10];

int varValue[MAX];

int isConst[MAX];

int numVars = 0;

int getVarIndex(char *name) {

    int i;

    for (i = 0; i < numVars; i++)

        if (strcmp(varName[i], name) == 0)

            return i;

    strcpy(varName[numVars], name);

    isConst[numVars] = 0;

    varValue[numVars] = 0;

    return numVars++;

}

int getOperandValue(char *tok, int *isKnown) {

    if (isdigit((unsigned char)tok[0]) ||

        (tok[0] == '-' && isdigit((unsigned char)tok[1]))) {

        *isKnown = 1;

        return atoi(tok);

    }
```

```
int idx = getVarIndex(tok);

*isKnown = isConst[idx];

return varValue[idx];

}

char *operandText(char *tok, int known, int value, char *buf) {

if (known)

sprintf(buf, "%d", value);

else

strcpy(buf, tok);

return buf;

}

int main() {

int n, i;

char stmt[50];

printf("Enter number of statements: ");

scanf("%d", &n);

getchar();

printf("Enter statements one per line\n");

printf("(formats: x=5 x=y+z print x) :\n\n");

printf("%-15s%-18s%s\n", "Statement", "Optimized Form", "Remark");

for (i = 0; i < n; i++) {

char left[10], right[40], op1[10], op2[10], opt[40], b1[15], b2[15];
```

```
int opPos, destIdx, known, known1, known2, v1, v2, result;

char opChar = 0;

fgets(stmt, sizeof(stmt), stdin);

stmt[strcspn(stmt, "\n")] = '\0';

if (strncmp(stmt, "print", 5) == 0) {

char v[10];

sscanf(stmt, "print %s", v);

int idx = getVarIndex(v);

if (isConst[idx]) {

sprintf(opt, "print %d", varValue[idx]);

printf("%-15s%-18s%s\n", stmt, opt, "Constant propagated");

} else {

printf("%-15s%-18s%s\n", stmt, stmt, "Value not constant");

}

continue;

}

sscanf(stmt, "%[^]=%s", left, right);

destIdx = getVarIndex(left);

opPos = -1;

for (int j = 1; right[j] != '\0'; j++) {

if (right[j] == '+' || right[j] == '-' || right[j] == '*' || right[j] == '/') {

opPos = j;


```

```
opChar = right[j];

break;

}

}

if (opPos == -1) {

/* simple assignment: dest = operand */

v1 = getOperandValue(right, &known);

varValue[destIdx] = v1;

isConst[destIdx] = known;

sprintf(opt, "%s=%d", left, v1);

printf("%-15s%-18s%s\n", stmt, known ? opt : stmt,

known ? "Constant propagated" : "Value not constant");

} else {

/* binary statement: dest = op1 OP op2 */

strncpy(op1, right, opPos);

op1[opPos] = '\0';

strcpy(op2, right + opPos + 1);

v1 = getOperandValue(op1, &known1);

v2 = getOperandValue(op2, &known2);

if (known1 && known2) {

switch (opChar) {

case '+': result = v1 + v2; break;
```

```
case '-': result = v1 - v2; break;

case '*': result = v1 * v2; break;

case '/': result = (v2 != 0) ? v1 / v2 : 0; break;

default: result = 0;

}

varValue[destIdx] = result;

isConst[destIdx] = 1;

sprintf(opt, "%s=%d", left, result);

printf("%-15s%-18s%s\n", stmt, opt, "Folded to constant");

} else if (known1 || known2) {

isConst[destIdx] = 0;

sprintf(opt, "%s=%s%c%s", left,

operandText(op1, known1, v1, b1), opChar,

operandText(op2, known2, v2, b2));

printf("%-15s%-18s%s\n", stmt, opt, "Operand propagated");

} else {

isConst[destIdx] = 0;

printf("%-15s%-18s%s\n", stmt, stmt, "Value not constant");

}

}

}

return 0;
```

```
}
```

EXPECTED OUTPUT:

Sample run for the statement sequence $x=4$, $y=x+6$, $z=y*2$, $w=z+p$, print w , print z (p is an unassigned/non-constant variable):

Enter number of statements: 6

Enter statements one per line

(formats: $x=5$ $x=y+z$ print x) :

Statement Optimized Form Remark

$x=4$ $x=4$ Constant propagated

$y=x+6$ $y=10$ Folded to constant

$z=y*2$ $z=20$ Folded to constant

$w=z+p$ $w=20+p$ Operand propagated

print w print w Value not constant

print z print 20 Constant propagated

MODEL VIVA QUESTIONS:

1. What is constant propagation, and how does it differ from constant folding?
2. Why is constant propagation classified as a data-flow optimization?
3. What happens to a variable's constant status when it is assigned different constant values along different control-flow paths?

RESULT:

A program was implemented in C to perform Constant Propagation on a given set of three-address statements, and it was verified that variables and expressions with compile-time-known values are correctly folded or propagated, while those depending on non-constant values are left unchanged.

EXP NO: 18**INTERMEDIATE CODE GENERATION FOR SIMPLE EXPRESSIONS****AIM:**

To implement, in C, a program that generates intermediate code (three-address code) for a given simple arithmetic expression, by first converting the infix expression to postfix form and then translating the postfix form into a sequence of three-address statements.

ALGORITHM:

1. Read the infix expression containing single-letter operands and the operators +, -, *, /, (and).
2. Convert the infix expression to postfix form using a stack, based on operator precedence.
3. Scan the postfix expression left to right, pushing each operand onto an operand stack.
4. On encountering an operator, pop its two operands, generate a new temporary, emit temp = operand1 operator operand2, and push the temporary back.
5. Repeat until the postfix expression is fully scanned; the final temporary holds the result. Display the postfix form and the generated three-address code.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX 50

char stack[MAX];

int top = -1;

void push(char c) { stack[++top] = c; }

char pop() { return stack[top--]; }
```

```
char peek() { return stack[top]; }

int precedence(char op) {

if (op == '+' || op == '-') return 1;

if (op == '*' || op == '/') return 2;

return 0;

}

int isOperator(char c) {

return c == '+' || c == '-' || c == '*' || c == '/';

}

void infixToPostfix(char *infix, char *postfix) {

int i, k = 0;

top = -1;

for (i = 0; infix[i] != '\0'; i++) {

char c = infix[i];

if (isalnum((unsigned char)c)) {

postfix[k++] = c;

} else if (c == '(') {

push(c);

} else if (c == ')') {

while (top != -1 && peek() != '(')

postfix[k++] = pop();

pop(); /* discard the '(' */
```

```
} else if (isOperator(c)) {  
  
while (top != -1 && precedence(peek()) >= precedence(c))  
  
postfix[k++] = pop();  
  
push(c);  
  
}  
  
}  
  
while (top != -1)  
  
postfix[k++] = pop();  
  
postfix[k] = '\0';  
  
}  
  
char opndStack[MAX][4];  
  
int opTop = -1;  
  
void pushOpnd(char *s) { strcpy(opndStack[++opTop], s); }  
  
char *popOpnd() { return opndStack[opTop--]; }  
  
int tempCount = 1;  
  
void generateTAC(char *postfix) {  
  
int i;  
  
char buf[4];  
  
opTop = -1;  
  
printf("\nThree-Address Code:\n");  
  
for (i = 0; postfix[i] != '\0'; i++) {  
  
char c = postfix[i];
```

```
if (isalnum((unsigned char)c)) {

buf[0] = c;

buf[1] = '\0';

pushOpnd(buf);

} else if (isOperator(c)) {

char *op2 = popOpnd();

char *op1 = popOpnd();

char temp[4];

sprintf(temp, "t%d", tempCount++);

printf("%s = %s %c %s\n", temp, op1, c, op2);

pushOpnd(temp);

}

}

printf("\nFinal result is held in %s\n", opndStack[opTop]);

}

int main() {

char infix[MAX], postfix[MAX];

printf("Enter an infix expression (single-letter operands, operators + - * / ( ) ): ");

scanf("%s", infix);

infixToPostfix(infix, postfix);

printf("\nPostfix expression: %s\n", postfix);

generateTAC(postfix);
```

```
return 0;
```

```
}
```

EXPECTED OUTPUT:

Sample run for the infix expression $a+b*c-d/e$:

Enter an infix expression (single-letter operands, operators + - * / ()): $a+b*c-d/e$

Postfix expression: $abc*+de/-$

Three-Address Code:

$t1 = b * c$

$t2 = a + t1$

$t3 = d / e$

$t4 = t2 - t3$

Final result is held in t4

MODEL VIVA QUESTIONS:

1. What is intermediate code, and why do compilers generate it instead of translating source code directly into target/machine code?
2. What is three-address code, and what are its typical forms of representation (quadruples, triples, indirect triples)?
3. Why is postfix (or an equivalent syntax tree) a convenient intermediate form for generating three-address code from an arithmetic expression?

RESULT:

A program was implemented in C to generate intermediate code for simple arithmetic expressions, and it was verified that a given infix expression is correctly converted to postfix form and then translated into an equivalent sequence of three-address statements using temporary variables.

EXP NO: 19

IMPLEMENTATION OF THE BACK END OF A COMPILER — THREE **ADDRESS CODE to 8086 ASSEMBLY LANGUAGE**

AIM:

To implement, in C, the back end of a compiler that translates a given sequence of three-address statements into 8086 assembly language instructions (MOV, ADD, SUB, MUL, DIV, CMP, JMP, conditional jumps) that can be assembled and run using an 8086 assembler.

ALGORITHM:

1. Read the three-address statements: assignments (dest=op1 OP op2 or dest=operand), unconditional jumps (goto L), conditional jumps (if x goto L), and labels (L:).
2. Scan all statements to collect label names (lines ending in ':') and variable names (every non-numeric, non-label identifier used), for declaring the data segment.
3. Emit the assembly skeleton: .MODEL, .STACK, and a .DATA section declaring each collected variable as a word (DW ?), followed by .CODE and the program prologue that loads DS.
4. Translate each statement in order: an assignment dest=op1+op2 (or -, *, /) becomes MOV AX,op1 followed by ADD/SUB/MOV BX+MUL/DIV, then MOV dest,AX; a simple copy becomes MOV AX,op1 then MOV dest,AX.
5. Translate goto L to JMP L, if x goto L to CMP x,0 followed by JNZ L, and a label line L: to an assembly label L:.
6. After all statements are translated, emit the program epilogue (MOV AH,4CH and INT 21H) and the END directive to complete an assembleable .ASM program.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX 20
```

```
#define LEN 50

char stmt[MAX][LEN];

int n;

char labelName[MAX][10];

int numLabels = 0;

char varName[MAX][10];

int numVars = 0;

int isLabelLine(char *s) {

int len = strlen(s);

return len > 1 && s[len - 1] == ':';

}

int isKnownLabel(char *tok) {

int i;

for (i = 0; i < numLabels; i++)

if (strcmp(labelName[i], tok) == 0)

return 1;

return 0;

}

void addVar(char *tok) {

int i;

if (isdigit((unsigned char)tok[0]) || (tok[0] == '-' && isdigit((unsigned char)tok[1])))

return; /* numeric literal, not a variable */
```

```
if (isKnownLabel(tok))

return; /* a label, not a variable */

for (i = 0; i < numVars; i++)

if (strcmp(varName[i], tok) == 0)

return; /* already recorded */

strcpy(varName[numVars++], tok);

}

/* ---- pass 1: collect label names ---- */

void collectLabels(void) {

int i;

for (i = 0; i < n; i++) {

if (isLabelLine(stmt[i])) {

char name[10];

strncpy(name, stmt[i], strlen(stmt[i]) - 1);

name[strlen(stmt[i]) - 1] = '\0';

strcpy(labelName[numLabels++], name);

}

}

}

/* ---- pass 2: collect variable names ---- */

void collectVars(void) {

int i;
```

```
for (i = 0; i < n; i++) {  
  
    char *s = stmt[i];  
  
    if (isLabelLine(s)) continue;  
  
    if (strncmp(s, "goto", 4) == 0) {  
  
        continue; /* no variable used */  
  
    } else if (strncmp(s, "if", 2) == 0) {  
  
        char cond[10], lbl[10];  
  
        sscanf(s, "if %s goto %s", cond, lbl);  
  
        addVar(cond);  
  
    } else {  
  
        char left[10], right[40], op1[10], op2[10];  
  
        int opPos = -1, j;  
  
        sscanf(s, "%[^]=%s", left, right);  
  
        addVar(left);  
  
        for (j = 1; right[j] != '\0'; j++) {  
  
            if (right[j] == '+' || right[j] == '-' || right[j] == '*' || right[j] == '/') {  
  
                opPos = j;  
  
                break;  
  
            }  
  
        }  
  
        if (opPos == -1) {  
  
            addVar(right);  
  
        }  
  
    }  
  
}
```

```
} else {

strcpy(op1, right, opPos); op1[opPos] = '\0';

strcpy(op2, right + opPos + 1);

addVar(op1);

addVar(op2);

}

}

}

}

/* ---- pass 3: emit 8086 assembly ---- */

void generateCode(void) {

int i;

printf(".MODEL SMALL\n");

printf(".STACK 100H\n");

printf(".DATA\n");

for (i = 0; i < numVars; i++)

printf(" %-6s DW ?\n", varName[i]);

printf(".CODE\n");

printf("MAIN PROC\n");

printf(" MOV AX, @DATA\n");

printf(" MOV DS, AX\n\n");

for (i = 0; i < n; i++) {
```

```
char *s = stmt[i];

if (isLabelLine(s)) {

printf("%s\n", s);

continue;

}

if (strncmp(s, "goto", 4) == 0) {

char lbl[10];

sscanf(s, "goto %s", lbl);

printf(" JMP %s\n", lbl);

} else if (strncmp(s, "if", 2) == 0) {

char cond[10], lbl[10];

sscanf(s, "if %s goto %s", cond, lbl);

printf(" CMP %s, 0\n", cond);

printf(" JNZ %s\n", lbl);

} else {

char left[10], right[40], op1[10], op2[10];

int opPos = -1, j;

char opChar = 0;

sscanf(s, "%[^]=%s", left, right);

for (j = 1; right[j] != '\0'; j++) {

if (right[j]=='+'||right[j]=='-'||right[j]=='*'||right[j]=='/') {

opPos = j;
```

```
opChar = right[j];

break;

}

}

if (opPos == -1) {

/* dest = operand */

printf(" MOV AX, %s\n", right);

printf(" MOV %s, AX\n", left);

} else {

strncpy(op1, right, opPos); op1[opPos] = '\0';

strcpy(op2, right + opPos + 1);

printf(" MOV AX, %s\n", op1);

switch (opChar) {

case '+':

printf(" ADD AX, %s\n", op2);

break;

case '-':

printf(" SUB AX, %s\n", op2);

break;

case '*':

printf(" MOV BX, %s\n", op2);

printf(" MUL BX\n");
```

```
break;

case '/':

printf(" MOV BX, %s\n", op2);

printf(" MOV DX, 0\n");

printf(" DIV BX\n");

break;

}

printf(" MOV %s, AX\n", left);

}

}

}

printf("\n MOV AH, 4CH\n");

printf(" INT 21H\n");

printf("MAIN ENDP\n");

printf("END MAIN\n");

}

int main() {

int i;

printf("Enter number of three-address statements: ");

scanf("%d", &n);

getchar();

printf("Enter statements one per line\n");
```

```
printf("(formats: x=5 x=y+z goto L1 if x goto L1 L1: )\n");

for (i = 0; i < n; i++)

fgets(stmt[i], LEN, stdin), stmt[i][strcspn(stmt[i], "\n")] = '\0';

collectLabels();

collectVars();

printf("\n8086 ASSEMBLY CODE\n");

printf("-----\n");

generateCode();

return 0;

}
```

EXPECTED OUTPUT:

Sample run for the three-address code $x=4$, $y=x+6$, $z=y*2$, if z goto $L1$, goto $L2$, $L1$:, $a=x-4$, $L2$::

Enter number of three-address statements: 8

Enter statements one per line

(formats: $x=5$ $x=y+z$ goto $L1$ if x goto $L1$ $L1$:)

$x=4$

$y=x+6$

$z=y*2$

if z goto $L1$

goto $L2$

$L1$:

$a=x-4$

L2:

8086 ASSEMBLY CODE

.MODEL SMALL

.STACK 100H

.DATA

x DW ?

y DW ?

z DW ?

a DW ?

.CODE

MAIN PROC

MOV AX, @DATA

MOV DS, AX

MOV AX, 4

MOV x, AX

MOV AX, x

ADD AX, 6

MOV y, AX

MOV AX, y

MOV BX, 2

MUL BX

```
MOV z, AX

CMP z, 0

JNZ L1

JMP L2

L1:

MOV AX, x

SUB AX, 4

MOV a, AX

L2:

MOV AH, 4CH

INT 21H

MAIN ENDP

END MAIN
```

MODEL VIVA QUESTIONS:

1. What is the role of the back end of a compiler, and how does it differ from the front end?
2. Why is a general-purpose register such as AX typically used as an accumulator when translating three-address code to assembly?
3. How would this simple code generator need to change to perform register allocation instead of always routing computations through AX?

RESULT:

A program was implemented in C as the back end of a compiler, translating a given sequence of three-address statements into 8086 assembly language instructions, and it was verified that the generated .ASM code correctly reflects the arithmetic, assignment, and control-flow structure of the input.